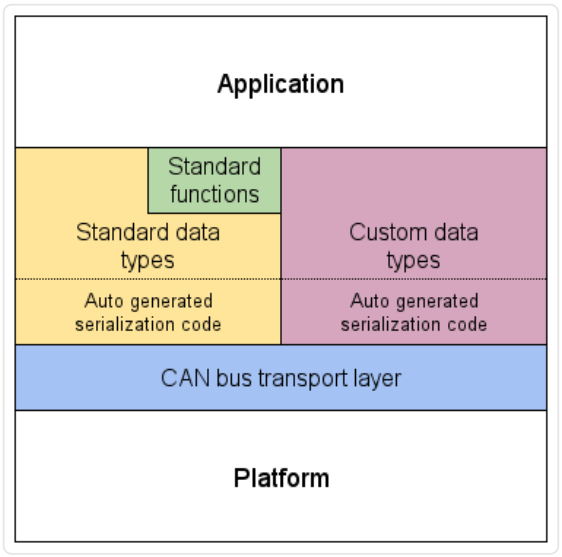
Application-level functions

The higher level concepts of DroneCAN are described in this section.



Node initialization

DroneCAN does not require that nodes undergo any specific initialization upon connecting to the bus - a node is free to begin functioning immediately once it is powered up. The only application-level function that every DroneCAN node must support is the periodic broadcasting of the node status message, which is documented next.

Node status reporting

Every DroneCAN node must report its status and presence by broadcasting messages of type `uavcan.protocol.NodeStatus`. This is the only data structure that DroneCAN nodes are required to support. All other application-level functions are considered optional.

Note that the ID of this message contains a long sequence of alternating 0 and 1 values when represented in binary, which facilitates automatic CAN bus bit rate detection.

The definition of the message is provided below.

`uavcan.protocol.NodeStatus`

Default data type ID: 341

```
#
# Abstract node status information.
#
# All UAVCAN nodes are required to publish this message periodically.
#

#
# Publication period may vary within these limits.
# It is NOT recommended to change it at run time.
#
uint16 MAX_BROADCASTING_PERIOD_MS = 1000
uint16 MIN_BROADCASTING_PERIOD_MS = 2

#
# If a node fails to publish this message in this amount of time, it should be considered offline.
#
uint16 OFFLINE_TIMEOUT_MS = 3000

#
# Uptime counter should never overflow.
# Other nodes may detect that a remote node has restarted when this value goes backwards.
#
uint32 uptime_sec

#
# Abstract node health.
#
uint2 HEALTH_OK         = 0     # The node is functioning properly.
uint2 HEALTH_WARNING    = 1     # A critical parameter went out of range or the node encountered a m
uint2 HEALTH_ERROR      = 2     # The node encountered a major failure.
uint2 HEALTH_CRITICAL   = 3     # The node suffered a fatal malfunction.
uint2 health

#
```


Specification

1. Introduction
2. Basic concepts
3. Data structure description language
4.1 CAN bus transport layer
4.2 UDP bus transport layer
4.3 MAVLink bus transport layer
4.4 CANFD bus transport layer
5. Application level conventions
6. Application level functions
7. List of standard data types
8. Hardware design recommendations
dsdl
  ardupilot
  uavcan

Contents

Application-level functions
  Node initialization
  Node status reporting
    `uavcan.protocol.NodeStatus`
  Node discovery
    `uavcan.protocol.GetNodeInfo`
    `uavcan.protocol.GetDataTypeInfo`
  Time synchronization
    Master
    Slave
    `uavcan.protocol.GlobalTimeSync`
  Node configuration
    `uavcan.protocol.param.ExecuteOp`
    `uavcan.protocol.param.GetSet`
    `uavcan.protocol.param.Empty`
    `uavcan.protocol.param.NumericVa`
    `uavcan.protocol.param.Value`
    Standard configuration parameters
  File transfer
    Firmware update
    `uavcan.protocol.file.BeginFirmwa`
    `uavcan.protocol.file.GetInfo`
    `uavcan.protocol.file.GetDirecto`
    `uavcan.protocol.file.Delete`
    `uavcan.protocol.file.Read`
    `uavcan.protocol.file.Write`
    `uavcan.protocol.file.EntryType`
    `uavcan.protocol.file.Error`
    `uavcan.protocol.file.Path`
  Debug features
    `uavcan.protocol.debug.KeyValue`

```
# participants that the sending node is about to shutdown. In this case other nodes will not
# have to wait OFFLINE_TIMEOUT_MS before they detect that the node is no longer available.
#
# Reserved values can be used in future revisions of the specification.
#
uint3 MODE_OPERATIONAL      = 0         # Normal operating mode.
uint3 MODE_INITIALIZATION   = 1         # Initialization is in progress; this mode is entered immedia
uint3 MODE_MAINTENANCE      = 2         # E.g. calibration, the bootloader is running, etc.
uint3 MODE_SOFTWARE_UPDATE  = 3         # New software/firmware is being loaded.
uint3 MODE_OFFLINE          = 7         # The node is no longer available.
uint3 mode

#
# Not used currently, keep zero when publishing, ignore when receiving.
#
uint3 sub_mode

#
# Optional, vendor-specific node status code, e.g. a fault code or a status bitmask.
#
uint16 vendor_specific_status_code
```

Node discovery

DroneCAN provides mechanisms to obtain the list of all nodes present in the network as well as detailed information about each node:

- Lists of all nodes that are connected to the bus can be created and maintained by listening for node status messages `uavcan.protocol.NodeStatus`.
- Extended information about each node can be requested using the services documented below.

Note that **it is highly recommended to support the service `uavcan.protocol.GetNodeInfo` in every node**, as it is vital for node discovery and identification.

`uavcan.protocol.GetNodeInfo`

Default data type ID: 1

```
#
# Full node info request.
# Note that all fields of the response section are byte-aligned.
#

---

#
# Current node status
#
NodeStatus status

#
# Version information shall not be changed while the node is running.
#
SoftwareVersion software_version
HardwareVersion hardware_version

#
# Human readable non-empty ASCII node name.
# Node name shall not be changed while the node is running.
# Empty string is not a valid node name.
# Allowed characters are: a-z (lowercase ASCII letters) 0-9 (decimal digits) . (dot) - (dash) _ (unde
# Node name is a reversed internet domain name (like Java packages), e.g. "com.manufacturer.project.p
#
uint8[<=80] name
```

`uavcan.protocol.GetDataTypeInfo`

Command shell access
`uavcan.protocol.AccessCommandShe`
Panic mode
`uavcan.protocol.Panic`
Dynamic node ID allocation
Allocatee-allocator exchanges
Non-redundant allocator
Redundant allocators

License

This work is licensed under a Creative Commons Attribution 4.0 International License.
Much of the content of this site is based upon prior work by Pavel Kirienko and the UAVCAN Development Team.

```
#
# Get the implementation details of a given data type.
#
# Request is interpreted as follows:
#  - If the field 'name' is empty, the fields 'kind' and 'id' will be used to identify the data type
#  - If the field 'name' is non-empty, it will be used to identify the data type; the
#    fields 'kind' and 'id' will be ignored.
#

uint16 id                   # Ignored if 'name' is non-empty
DataTypeKind kind           # Ignored if 'name' is non-empty

uint8[<=80] name            # Full data type name, e.g. "uavcan.protocol.GetDataTypeInfo"

---

uint64 signature            # Data type signature; valid only if the data type is known (see FLAG_KNO

uint16 id                   # Valid only if the data type is known (see FLAG_KNOWN)
DataTypeKind kind           # Ditto

uint8 FLAG_KNOWN      = 1   # This data type is defined
uint8 FLAG_SUBSCRIBED = 2   # Subscribed to messages of this type
uint8 FLAG_PUBLISHING = 4   # Publishing messages of this type
uint8 FLAG_SERVING    = 8   # Providing service of this type
uint8 flags

uint8[<=80] name            # Full data type name
```

Time synchronization

DroneCAN supports network-wide precise time synchronization with a resolution of up to 1 CAN bus bit period (i.e., 1 microsecond for 1 Mbps CAN bit rate), assuming that CAN frame timestamping is supported by the hardware. The algorithm can also function in the absence of hardware support for timestamping, although its performance will be degraded.

The time synchronization approach is based on the work “Implementing a Distributed High-Resolution Real-Time Clock using the CAN-Bus” (M. Gergeleit and H. Streich). The general idea of the algorithm is to have one or more nodes that periodically broadcast a message of type `uavcan.protocol.GlobalTimeSync` (definition is provided below) containing the exact timestamp of the previous transmission of this message. A node that performs a periodic broadcast of this message is referred to as a *time synchronization master*, whereas a node that synchronizes its time with the master is referred to as a *time synchronization slave*.

Note that this algorithm only allows to precisely estimate the phase difference between the given slave and the master it is synchronized with. DroneCAN does not define the algorithm for clock speed/phase adjustment, which is entirely implementation defined.

The following constants are defined for the time synchronization algorithm:

- T_{max} - maximum broadcast interval for a given master.
- T_{min} - minimum broadcast interval for a given master.
- $T_{timeout}$ - if the master was not broadcasting the time synchronization message for this amount of time, all slaves shall switch to the next active master with the highest priority.

The network may accommodate more than one time synchronization master working at the same time. In this case, only the master with the lowest node ID should be active; other masters should become passive by means of stopping broadcasting time synchronization messages, and they must synchronize with the active master instead. If the currently active master was not broadcasting time synchronization messages for the duration of $T_{timeout}$, the next master with the highest priority becomes active instead, and all slaves will synchronize with it. When a higher priority master appears in the network, all other lower-priority masters should become passive, and all slaves will synchronize with the new master immediately.

The message `uavcan.protocol.GlobalTimeSync` contains the exact timestamp of the previous transmission of this message. If the previous message was not yet transmitted, or if it was transmitted

It is recommended to refer to the existing DroneCAN implementations for reference.

Master

The following pseudocode describes the logic of a time synchronization master.

```
// State variables:
transfer_id := 0;
previous_tx_timestamp[NUM_IFACES];
previous_broadcast_timestamp;

// This function broadcasts a message with a specified Transfer ID using only one iface:
function broadcastMessage(transfer_id, iface_index, msg);

// This function returns the current value of a monotonic clock (the clock that doesn't change phase
function getMonotonicTime();

// This callback is invoked when the CAN driver completes transmission of a time sync message
// The tx_timestamp argument contains the exact timestamp when the CAN frame was delivered to the bus
function messageTxTimestampCallback(iface_index, tx_timestamp)
{
    previous_tx_timestamp[iface_index] := tx_timestamp;
}

// Publishes the message of type uavcan.protocol.GlobalTimeSync to each available interface
function broadcastTimeSync()
{
    current_time := getMonotonicTime();

    if (current_time - previous_broadcast_timestamp < MIN_PUBLICATION_PERIOD)
    {
        return;     // Rate limiting
    }

    if (current_time - previous_broadcast_timestamp > MAX_PUBLICATION_PERIOD)
    {
        for (i := 0; i < NUM_IFACES; i++)
        {
            previous_tx_timestamp[i] := 0;
        }
    }

    previous_broadcast_timestamp := current_time;

    message := uavcan.protocol.GlobalTimeSync();

    for (i := 0; i < NUM_IFACES; i++)
    {
        message.previous_transmission_timestamp_usec := previous_tx_timestamp[i];
        previous_tx_timestamp[i] := 0;
        broadcastMessage(transfer_id, i, message);
    }

    transfer_id++; // Overflow must be handled correctly
}
```

Slave

The following pseudocode describes the logic of a time synchronization slave.

```
// State variables:
previous_rx_real_timestamp := 0;               // This time is being synchronized
previous_rx_monotonic_timestamp := 0;     // This is the monotonic time (doesn't jump or change rate)
previous_transfer_id := 0;
state := STATE_UPDATE;       // STATE_UPDATE, STATE_ADJUST
master_node_id := -1;        // Invalid value
iface_index := -1;           // Invalid value

// This function performs local clock adjustment:
function adjustLocalTime(phase_error);

function adjust(message)
{
```

```
    state := STATE_UPDATE;
}

function update(message)
{
    // Message is assumed to have two timestamps:
    //   Real - sampled from the clock that is being synchronized
    //   Monotonic - clock that never jumps and never changes rate
    previous_rx_real_timestamp := message.rx_real_timestamp;
    previous_rx_monotonic_timestamp := message.rx_monotonic_timestamp;
    master_node_id := message.source_node_id;
    iface_index := message.iface_index;
    previous_transfer_id := message.transfer_id;
    state := STATE_ADJUST;
}

// Accepts the message of type uavcan.protocol.GlobalTimeSync (please refer to the DSDL definition)
function handleReceivedTimeSyncMessage(message)
{
    time_since_previous_msg := message.monotonic_timestamp - previous_rx_monotonic_timestamp;

    // Resolving the state flags:
    needs_init := (master_node_id < 0) or (iface_index < 0);
    switch_master := message.source_node_id < master_node_id;
    publisher_timed_out := time_since_previous_msg > PUBLISHER_TIMEOUT;

    if (needs_init or switch_master or publisher_timed_out)
    {
        update(message);
    }
    else if ((message.iface_index == iface_index) and (message.source_node_id == master_node_id))
    {
        // Revert the state to STATE_UPDATE if needed
        if (state == STATE_ADJUST)
        {
            msg_invalid := message.previous_transmission_timestamp_usec == 0;
            wrong_tid := message.transfer_id != (previous_transfer_id + 1);    // Overflow must be ha
            wrong_timing := time_since_previous_msg > MAX_PUBLICATION_PERIOD;
            if (msg_invalid or wrong_tid or wrong_timing)
            {
                state := STATE_UPDATE;
            }
        }
        // Handle the current state
        if (state == STATE_ADJUST)
        {
            adjust(message);
        }
        else
        {
            update(message);
        }
    }
    else
    {
        ; // Ignore this message
    }
}
```

`uavcan.protocol.GlobalTimeSync`

Default data type ID: 4

```
#
# Global time synchronization.
# Any node that publishes timestamped data must use this time reference.
#
# Please refer to the specification to learn about the synchronization algorithm.
#

#
# Broadcasting period must be within this range.
#
uint16 MAX_BROADCASTING_PERIOD_MS = 1100            # Milliseconds
```

```
# Synchronization slaves may switch to a new source if the current master was silent for this amount
#
uint16 RECOMMENDED_BROADCASTER_TIMEOUT_MS = 2200    # Milliseconds

#
# Time in microseconds when the PREVIOUS GlobalTimeSync message was transmitted.
# If this message is the first one, this field must be zero.
#
truncated uint56 previous_transmission_timestamp_usec # Microseconds
```

Node configuration

DroneCAN defines standard services for management of remote node's configuration parameters. Support for these services is not mandatory but is highly recommended. The services are as follows:

- `uavcan.protocol.param.GetSet` - gets or sets a single configuration parameter value, either by name or by index.
- `uavcan.protocol.param.ExecuteOpcode` - allows control of the node configuration, including saving the configuration into the non-volatile memory, or resetting the configuration to default settings.
- `uavcan.protocol.RestartNode` - restarts a node remotely. Some nodes may require a restart before new configuration parameters can be applied.

In some cases, a node may require more complex configuration than can be conveniently managed via these services. If this is the case, the recommendation is to manage the node's configuration through configuration files accessible via the standard file management services (documented in this section).

`uavcan.protocol.param.ExecuteOpcode`

Default data type ID: 10

```
#
# Service to control the node configuration.
#

#
# SAVE operation instructs the remote node to save the current configuration parameters into a non-vo
# storage. The node may require a restart in order for some changes to take effect.
#
# ERASE operation instructs the remote node to clear its configuration storage and reinitialize the p
# with their default values. The node may require a restart in order for some changes to take effect
#
# Other opcodes may be added in the future (for example, an opcode for switching between multiple cor
#
uint8 OPCODE_SAVE  = 0  # Save all parameters to non-volatile storage.
uint8 OPCODE_ERASE = 1  # Clear the non-volatile storage; some changes may take effect only after reb
uint8 opcode

#
# Reserved, keep zero.
#
int48 argument

---

#
# If 'ok' (the field below) is true, this value is not used and must be kept zero.
# If 'ok' is false, this value may contain error code. Error code constants may be defined in the fut
#
int48 argument

#
# True if the operation has been performed successfully, false otherwise.
#
bool ok
```

`uavcan.protocol.param.GetSet`

```
#
# Get or set a parameter by name or by index.
# Note that access by index should only be used to retrieve the list of parameters; it is highly
# discouraged to use it for anything else, because persistent ordering is not guaranteed.
#

#
# Index of the parameter starting from 0; ignored if name is nonempty.
# Use index only to retrieve the list of parameters.
# Parameter ordering must be well defined (e.g. alphabetical, or any other stable ordering),
# in order for the index access to work.
#
uint13 index

#
# If set - parameter will be assigned this value, then the new value will be returned.
# If not set - current parameter value will be returned.
# Refer to the definition of Value for details.
#
Value value

#
# Name of the parameter; always preferred over index if nonempty.
#
uint8[<=92] name

---

void5

#
# Actual parameter value.
#
# For set requests, it should contain the actual parameter value after the set request was
# executed. The objective is to let the client know if the value could not be updated, e.g.
# due to its range violation, etc.
#
# Empty value (and/or empty name) indicates that there is no such parameter.
#
Value value

void5
Value default_value    # Optional

void6
NumericValue max_value # Optional, not applicable for bool/string

void6
NumericValue min_value # Optional, not applicable for bool/string

#
# Empty name (and/or empty value) in response indicates that there is no such parameter.
#
uint8[<=92] name
```

uavcan.protocol.param.Empty

```
#
# Ex nihilo nihil fit.
#
```

uavcan.protocol.param.NumericValue

```
#
# Numeric-only value.
#
# This is a union, which means that this structure can contain either one of the fields below.
# The structure is prefixed with tag - a selector value that indicates which particular field is enco
#
```

```
int64   integer_value
float32 real_value
```

`uavcan.protocol.param.Value`

```
#
# Single parameter value.
#
# This is a union, which means that this structure can contain either one of the fields below.
# The structure is prefixed with tag - a selector value that indicates which particular field is enco
#

@union                          # Tag is 3 bit long, so outer structure has 5-bit prefix to ensure pr

Empty empty                     # Empty field, used to represent an undefined value.

int64        integer_value
float32      real_value         # 32-bit type is used to simplify implementation on low-end systems
uint8        boolean_value      # 8-bit value is used for alignment reasons
uint8[<=128] string_value       # Length prefix is exactly one byte long, which ensures proper alignm
```

Standard configuration parameters

There are some configuration parameters that are common for most DroneCAN nodes. Examples of such common parameters include message publication frequencies, non-default data type ID settings, local node ID, etc. The DroneCAN specification improves compatibility by providing the following naming conventions for DroneCAN-related configuration parameters. Following these conventions is highly encouraged, but not mandatory.

As can be seen below, all standard DroneCAN-related parameters share the same prefix `uavcan.`.

Data type ID

Parameter name: `uavcan.dtid-X`, where **X** stands for the full data type name, e.g. `uavcan.dtid-uavcan.protocol.NodeStatus`.

This parameter configures the data type ID value for a given data type.

Message publication period

Parameter name: `uavcan.pubp-X`, where **X** stands for the full data type name; e.g. `uavcan.pubp-uavcan.protocol.NodeStatus`.

This parameter configures the publication period for a given data type, in integer number of microseconds. Zero value means that publication should be disabled.

Transfer priority

Parameter name: `uavcan.prio-X`, where **X** stands for the full data type name, e.g. `uavcan.prio-uavcan.protocol.NodeStatus`.

This parameter configures the transport priority level that will be used when publishing messages or calling services of a given data type.

Node ID

Parameter name: `uavcan.node_id`.

This parameter configures ID of the local node. Zero means that the node ID is unconfigured, which may prompt the node to resort to dynamic node ID allocation after startup.

CAN bus bit rate

Parameter name: `uavcan.bit_rate`.

This parameter configures CAN bus bit rate. Zero value should trigger automatic bit rate detection, which should be the default option. Please refer to the hardware design recommendations for

Parameter name: `uavcan.id-X-Y`, where **X** is namespace name; **Y** is ID field name.

Some DroneCAN messages (standard and possibly vendor-specific ones) use special fields that identify the instance of a certain function - *ID fields*. For example, messages related to actuator control use fields named `actuator_id`, some sensor messages use fields named `sensor_id`, etc. In order to improve compatibility, the specification offers a naming convention for parameters that define the values used in ID fields.

Given messages located in the namespace **X** that share an ID field named **Y**, the corresponding parameter name would be `uavcan.id-X-Y`. For example, the parameter for the field `esc_index` that is used in the message `uavcan.equipment.esc.Status` and that defines the array index in `uavcan.equipment.esc.RawCommand`, will be named as follows:

```
uavcan.id-uavcan.equipment.esc-esc_index
```

In the case that an ID field is shared across different namespaces, then the most common outer shared namespace should be used as **X**. This is not the case for any of the standard messages, so an example cannot be provided.

In the case that an ID field is used in the standard namespace (`uavcan.*`) and in some vendor-specific namespaces at the same time, the prefix should be used as though the ID field was used only in the standard namespace.

File transfer

File transfer is a very generic feature of DroneCAN, that allows access to the file system on remote nodes. The feature is based upon a set of DroneCAN services that are listed below.

Firmware update

In terms of DroneCAN, firmware update is a special case of file transfer. The process of firmware update involves two or three nodes:

- The node that initiates the process of firmware update, or *updater*.
- The node that provides access to the firmware file, or *file server*. In most cases, the updater will be acting as a file server and only two nodes are involved.
- The node that is being updated, or *updatee*.

The process can be described as follows:

1. The updater decides that a certain node (the *updatee*) should be updated.
2. The updater invokes the service `uavcan.protocol.file.BeginFirmwareUpdate` on the updatee. The information about the location of the firmware file will be passed to the updatee via the service request.
3. If the updatee chooses to accept the update request, it performs initialization procedures as required by its implementation (e.g., rebooting into the bootloader, etc).
4. The updatee receives new firmware file from the file server using information received via the service request above.
5. The updatee completes the update and restarts.

Typically, the updatee will also resort to the dynamic node ID allocation process, which is documented in this section.

`uavcan.protocol.file.BeginFirmwareUpdate`

Default data type ID: 40

```
#
# This service initiates firmware update on a remote node.
#
# The node that is being updated (slave) will retrieve the firmware image file 'image_file_remote_pat
# 'source_node_id' using the file read service, then it will update the firmware and reboot.
#
```

```
# If the slave node accepts this request, the initiator will get a response immediately, before the u
# actually begins.
#
# While the firmware is being updated, the slave should set its mode (uavcan.protocol.NodeStatus.mode
# MODE_SOFTWARE_UPDATE.
#

uint8 source_node_id         # If this field is zero, the caller's Node ID will be used instead.
Path image_file_remote_path

---

#
# Other error codes may be added in the future.
#
uint8 ERROR_OK               = 0
uint8 ERROR_INVALID_MODE     = 1    # Cannot perform the update in the current operating mode or stat
uint8 ERROR_IN_PROGRESS      = 2    # Firmware update is already in progress, and the slave doesn't w
uint8 ERROR_UNKNOWN          = 255
uint8 error

uint8[<128] optional_error_message   # Detailed description of the error.
```

`uavcan.protocol.file.GetInfo`

Default data type ID: 45

```
#
# Request info about a remote file system entry (file, directory, etc).
#

Path path

---

#
# File size in bytes.
# Should be set to zero for directories.
#
uint40 size

Error error

EntryType entry_type
```

`uavcan.protocol.file.GetDirectoryEntryInfo`

Default data type ID: 46

```
#
# This service can be used to retrieve a remote directory listing, one entry per request.
#
# The client should query each entry independently, iterating 'entry_index' from 0 until the last ent
# in which case the server will report that there is no such entry (via the fields 'entry_type' and
#
# The entry_index shall be applied to the ordered list of directory entries (e.g. alphabetically orde
# sorting criteria does not matter as long as it provides the same ordering for subsequent service ca
#

uint32 entry_index

Path directory_path

---

Error error

EntryType entry_type
```

uavcan.protocol.file.Delete

Default data type ID: 47

```
#
# Delete remote file system entry.
# If the remote entry is a directory, all nested entries will be removed too.
#

Path path

---

Error error
```

uavcan.protocol.file.Read

Default data type ID: 48

```
#
# Read file from a remote node.
#
# There are two possible outcomes of a successful service call:
#  1. Data array size equals its capacity. This means that the end of the file is not reached yet.
#  2. Data array size is less than its capacity, possibly zero. This means that the end of file is re
#
# Thus, if the client needs to fetch the entire file, it should repeatedly call this service while in
# offset, until incomplete data is returned.
#
# If the object pointed by 'path' cannot be read (e.g. it is a directory or it does not exist), appro
# will be returned, and data array will be empty.
#

uint40 offset

Path path

---

Error error

uint8[<=256] data
```

uavcan.protocol.file.Write

Default data type ID: 49

```
#
# Write into a remote file.
# The server shall place the contents of the field 'data' into the file pointed by 'path' at the offs
# the field 'offset'.
#
# When writing a file, the client should repeatedly call this service with data while advancing offse
# is written completely. When write is complete, the client shall call the service one last time, wit
# set to the size of the file and with the data field empty, which will signal the server that the wr
# complete.
#
# When the write operation is complete, the server shall truncate the resulting file past the specif
#
# Server implementation advice:
# It is recommended to implement proper handling of concurrent writes to the same file from different
# example by means of creating a staging area for uncompleted writes (like FTP servers do).
#

uint40 offset

Path path
```

```
---

Error error
```

`uavcan.protocol.file.EntryType`

```
#
# Nested type.
# Represents the type of the file system entry (e.g. file or directory).
# If such entry does not exist, 'flags' must be set to zero.
#

uint8 FLAG_FILE      = 1        # Excludes FLAG_DIRECTORY
uint8 FLAG_DIRECTORY = 2        # Excludes FLAG_FILE
uint8 FLAG_SYMLINK   = 4        # Link target is either FLAG_FILE or FLAG_DIRECTORY
uint8 FLAG_READABLE  = 8
uint8 FLAG_WRITEABLE = 16

uint8 flags
```

`uavcan.protocol.file.Error`

```
#
# Nested type.
# File operation result code.
#

int16 OK                = 0
int16 UNKNOWN_ERROR     = 32767

int16 NOT_FOUND         = 2
int16 IO_ERROR          = 5
int16 ACCESS_DENIED     = 13
int16 IS_DIRECTORY      = 21 # I.e. attempt to read/write on a path that points to a directory
int16 INVALID_VALUE     = 22 # E.g. file name is not valid for the target file system
int16 FILE_TOO_LARGE    = 27
int16 OUT_OF_SPACE      = 28
int16 NOT_IMPLEMENTED   = 38

int16 value
```

`uavcan.protocol.file.Path`

```
#
# Nested type.
#
# File system path in UTF8.
#
# The only valid separator is forward slash.
#

uint8 SEPARATOR = '/'

uint8[<=200] path
```

Debug features

The following messages are designed to facilitate debugging and to provide means of reporting events in a human-readable representation.

`uavcan.protocol.debug.KeyValue`

```
#
# Generic named parameter (key/value pair).
#

#
# Integers are exactly representable in the range (-2^24, 2^24) which is (-16'777'216, 16'777'216).
#
float32 value

#
# Tail array optimization is enabled, so if key length does not exceed 3 characters, the whole
# message can fit into one CAN frame. The message always fits into one CAN FD frame.
#
uint8[<=58] key
```

`uavcan.protocol.debug.LogMessage`

Default data type ID: 16383

```
#
# Generic log message.
# All items are byte aligned.
#

LogLevel level
uint8[<=31] source
uint8[<=90] text
```

`uavcan.protocol.debug.LogLevel`

```
#
# Log message severity
#

uint3 DEBUG    = 0
uint3 INFO     = 1
uint3 WARNING  = 2
uint3 ERROR    = 3
uint3 value
```

Command shell access

The following service allows execution of arbitrary commands on a remote node via direct access to its internal command shell.

`uavcan.protocol.AccessCommandShell`

Default data type ID: 6

```
#
# THIS DEFINITION IS SUBJECT TO CHANGE.
#
# This service allows to execute arbitrary commands on the remote node's internal system shell.
#
# Essentially, this service mimics a typical terminal emulator, with one text input (stdin) and two t
# outputs (stdout and stderr). When there's no process running, the input is directed into the termir
# handler itself, which interprets it. If there's a process running, the input will be directed into
# stdin of the running process. It is possible to forcefully return the terminal into a known state b
# means of setting the reset flag (see below), in which case the terminal will kill all of the child
# processes, if any, and return into the initial idle state.
#
# The server is assumed to allocate one independent terminal instance per client, so that different c
# can execute commands without interfering with each other.
#
```

```
uint8 NEWLINE = '\n'

#
# The server is required to keep the result of the last executed command for at least this time.
# When this time expires, the server may remove the results in order to reclaim the memory, but it
# is not guaranteed. Hence, the clients must retrieve the results in this amount of time.
#
uint8 MIN_OUTPUT_LIFETIME_SEC = 10

#
# These flags control the shell and command execution.
#
uint8 FLAG_RESET_SHELL          = 1     # Restarts the shell instance anew; may or may not imply CLEA
uint8 FLAG_CLEAR_OUTPUT_BUFFERS = 2     # Makes stdout and stderr buffers empty
uint8 FLAG_READ_STDOUT          = 64    # Output will contain stdout
uint8 FLAG_READ_STDERR          = 128   # Output will be extended with stderr
uint8 flags

#
# If the shell is idle, it will interpret this string.
# If there's a process running, this string will be piped into its stdin.
#
# If RESET_SHELL is set, new input will be interpreted by the shell immediately.
#
uint8[<=128] input

---

#
# Exit status of the last executed process, or error code of the shell itself.
# Default value is zero.
#
int32 last_exit_status

#
# These flags indicate the status of the shell.
#
uint8 FLAG_RUNNING              = 1     # The shell is currently running a process; stdin/out/err are
uint8 FLAG_SHELL_ERROR          = 2     # Exit status contains error code, output contains text (e.g
uint8 FLAG_HAS_PENDING_STDOUT   = 64    # There is more stdout to read
uint8 FLAG_HAS_PENDING_STDERR   = 128   # There is more stderr to read
uint8 flags

#
# In case of a shell error, this string may contain ASCII string explaining the nature of the error.
# Otherwise, if stdout read is requested, this string will contain stdout data. If stderr read is req
# this string will contain stderr data. If both stdout and stderr read is requested, this string will
# with stdout and end with stderr, with no separator in between.
#
uint8[<=256] output
```

Panic mode

The panic message allows the broadcaster to quickly shut down the system in the event of an emergency.

`uavcan.protocol.Panic`

Default data type ID: 5

```
#
# This message may be published periodically to inform network participants that the system has encou
# an unrecoverable fault and is not capable of further operation.
#
# Nodes that are expected to react to this message should wait for at least MIN_MESSAGES subsequent m
# with any reason text from any sender published with the interval no higher than MAX_INTERVAL_MS bef
# undertaking any emergency actions.
#

uint8 MIN_MESSAGES = 3
```

```
# Short description that would fit a single CAN frame.
#
uint8[<=7] reason_text
```

Dynamic node ID allocation

In order to be able to operate in a DroneCAN network, a node must have a node ID that is unique within the network. Typically, a valid node ID can be configured manually for each node; however, in certain use cases the manual approach is either undesirable or impossible, therefore DroneCAN defines the high-level feature of dynamic node ID allocation, that allows nodes to obtain a node ID value automatically upon connection to the network.

Dynamic node ID allocation combined with automatic CAN bus bit rate detection makes it easy to implement nodes that can join any DroneCAN network without any manual configuration. These sorts of nodes are referred to as *plug-and-play nodes*.

A dynamically allocated node ID cannot be persistent. This means that if a node is configured to use a dynamic node ID, it *must* perform a new allocation every time it starts or reboots.

The process of dynamic node ID allocation always involves two types of nodes: *allocators*, which serve allocation requests; and *allocatees*, which request dynamic node ID from allocators. A DroneCAN network may implement the following configurations of allocators:

- Zero allocators, in which case the feature of dynamic node ID allocation will not be available.
- One allocator, in which case the feature of dynamic node ID allocation will become unavailable if the allocator fails. In this configuration, the role of the allocator can be performed even by a very resource-constrained system, e.g. a low-end microcontroller.
- Three allocators, in which case the allocators will be using a replicated state via a distributed consensus algorithm. In this configuration, the network can tolerate the loss of one allocator and continue to serve allocation requests. This configuration requires that the allocators to maintain large data structures for the purposes of the distributed consensus algorithm, and may therefore require a slightly more sophisticated computational platform, e.g. a high-end microcontroller.
- Five allocators, is the same as the three allocator configuration except that the network can tolerate the loss of two allocators and still continue to serve allocation requests.

In order to get a dynamic node ID, each allocatee must have a globally unique 128-bit integer identifier, known as *unique ID*. This is the same value that is used in the field `unique_id` of the data type `uavcan.protocol.HardwareVersion`. Every node that requires a dynamic ID allocation must support the service `uavcan.protocol.GetNodeInfo`, and the nodes must use the same unique ID value during dynamic node ID allocation and when responding to `uavcan.protocol.GetNodeInfo` requests.

During dynamic allocation, the allocatee communicates its unique ID to the allocator (or allocators), which then use it to produce an appropriate allocation response. Unique ID values are kept by allocators in *allocation tables* - data structures that contain mappings between unique ID and corresponding node ID values. Allocation tables are write-only data structures that can only grow. Once a new allocatee has requested a node ID, its unique ID will be recorded into the allocation table, and all subsequent allocation requests from the same allocatee will be served with the same node ID value.

In configurations with redundant allocators, every allocator maintains a replica of the same allocation table (a DroneCAN network cannot contain more than one allocation table, regardless of the number of allocators employed). While the allocation table is write-only data structure that can only grow, it is still possible to wipe the table completely, forcing the allocators to forget known nodes and perform all following allocations anew.

In the context of this chapter, nodes that are using dynamic node ID will be referred to as *dynamic nodes*, and nodes that are using manually-configured node ID will be referred to as *static nodes*. It is assumed that in most cases, allocators will be static nodes themselves (since there's no other authority on the network that can grant dynamic node ID, allocators will not be able to dynamically allocate themselves). Excepting allocators, it is not recommended to mix dynamic and static nodes on the same network; i.e., normally, a DroneCAN network should contain either all static nodes, or all dynamic nodes (except allocators). In case if this recommendation cannot be followed, the following rules of safe co-existence of dynamic nodes with static nodes must be considered:

them, i.e. these static nodes are already in the allocation table.
3. A new static node (i.e. a node that does not meet the above condition) can be connected to the bus only if:
   1. New dynamic allocations are not happening at the moment.
   2. The allocators are capable of serving new allocations.

As can be inferred from the above, the process of dynamic node ID allocation involves up to two types of communications:

- *Allocatee-allocator* - this communication is used when an allocatee requests a dynamic node ID from the allocator (allocators), and when the allocator (allocators) transmits a response back to the allocatee. This communication is invariant to the allocator configuration used, i.e., the allocatees are not aware of how many allocators are available on the network and how they are configured.
- *Allocator-allocator* - this communication is used by allocators for the purpose of maintenance of the replicated allocation table and for other needs of the distributed consensus algorithm. Allocatees are completely isolated and unaware of these exchanges. This communication is not applicable for the single-allocator configuration.

Allocatee-allocator exchanges

Allocatee-allocator exchanges are performed using only one message type - `uavcan.protocol.dynamic_node_id.Allocation`. Allocators use it with regular message broadcast transfers; allocatees use it with anonymous message transfers. The specification and usage info for this data type is provided below.

The general idea of the allocatee-allocator exchanges is that the allocatee communicates to the allocator its unique ID and, if applicable, the preferred node ID value, using anonymous message transfers of type `uavcan.protocol.dynamic_node_id.Allocation`. The allocator performs the allocation and sends a response using the same message type, where the field for unique ID is populated with the unique ID of the requesting node and the field for node ID is populated with the allocated node ID. Note that since the allocator that serves the allocation always has a node ID, it is free to use multi-frame transfers, therefore the allocator can directly send the response using a single message transfer. The allocatees, however, are restricted to single-frame transfers, due to limitations of anonymous message transfers. Therefore, the allocatees send their unique ID to the allocator using three single-frame transfers, where the first transfer contains the first part of their unique ID, second transfer contains the continuation, and the last transfer contains the last few bytes of the unique ID. The details are provided in the DSDL description of the message type.

The specification of the data type contains a description of the exchange protocol on the side of allocatee. On the allocator's side the algorithm should be implemented as shown in the following pseudocode.

Please note that the pseudocode refers to a function named `canPublishFollowupAllocationResponse()`, which is only applicable in the case of a redundant allocator configuration. It evaluates the current state of the distributed consensus and decides whether the current node is allowed to engage in allocation exchanges. The logic of this function will be reviewed in the chapter dedicated to redundant allocators. In the non-redundant allocator configuration, this function will always return true, meaning that the allocator is always allowed to engage in allocation exchanges.

```
// Constants:
InvalidStage = 0;

// State variables:
last_message_timestamp;
current_unique_id;

// This function will be invoked when a complete unique ID is received.
// Typically, the actual allocation will be carried out in this function.
function handleAllocationRequest(unique_id, preferred_node_id);

// This function is only applicable in a configuration with redundant allocators.
// Its return value depends on the current state of the distributed consensus algorithm.
// Please refer to the allocator-allocator communication logic for details.
// In the non-redundant configuration this function will always return true.
function canPublishFollowupAllocationResponse();
```

```
{
    if ((msg.unique_id.size() != MAX_LENGTH_OF_UNIQUE_ID_IN_REQUEST) &&
        (msg.unique_id.size() != (msg.unique_id.capacity() - MAX_LENGTH_OF_UNIQUE_ID_IN_REQUEST * 2U)
        (msg.unique_id.size() != msg.unique_id.capacity()))     // For CAN FD
    {
        return InvalidStage;
    }
    if (msg.first_part_of_unique_id)
    {
        return 1;       // Note that CAN FD frames can deliver the unique ID in one stage!
    }
    if (msg.unique_id.size() == MAX_LENGTH_OF_UNIQUE_ID_IN_REQUEST)
    {
        return 2;
    }
    if (msg.unique_id.size() < MAX_LENGTH_OF_UNIQUE_ID_IN_REQUEST)
    {
        return 3;
    }
    return InvalidStage;
}

// This is an internal function; see below.
function getExpectedStage()
{
    if (current_unique_id.empty())
    {
        return 1;
    }
    if (current_unique_id.size() >= (MAX_LENGTH_OF_UNIQUE_ID_IN_REQUEST * 2))
    {
        return 3;
    }
    if (current_unique_id.size() >= MAX_LENGTH_OF_UNIQUE_ID_IN_REQUEST)
    {
        return 2;
    }
    return InvalidStage;
}

// This function is invoked when the allocator receives a message of type uavcan.protocol.dynamic_noc
function handleAllocation(msg)
{
    if (!msg.isAnonymousTransfer())
    {
        return;         // This is a response from another allocator, ignore
    }

    // Reset the expected stage on timeout
    if (msg.getMonotonicTimestamp() > (last_message_timestamp + FOLLOWUP_TIMEOUT))
    {
        current_unique_id.clear();
    }

    // Checking if request stage matches the expected stage
    request_stage = detectRequestStage(msg);
    if (request_stage == InvalidStage)
    {
        return;             // Malformed request - ignore without resetting
    }

    if (request_stage != getExpectedStage())
    {
        return;             // Ignore - stage mismatch
    }

    if (msg.unique_id.size() > current_unique_id.capacity() - current_unique_id.size())
    {
        return;             // Malformed request
    }

    // Updating the local state
    for (i = 0; i < msg.unique_id.size(); i++)
    {
        current_unique_id.push_back(msg.unique_id[i]);
    }
```

```
        // Proceeding with allocation.
        handleAllocationRequest(current_unique_id, msg.node_id);
        current_unique_id.clear();
    }
    else
    {
        // Publishing the follow-up if possible.
        if (canPublishFollowupAllocationResponse())
        {
            msg = uavcan.protocol.dynamic_node_id.Allocation();
            msg.unique_id = current_unique_id;
            broadcast(msg);
        }
        else
        {
            current_unique_id.clear();
        }
    }

    // It is important to update the timestamp only if the request has been processed successfully.
    last_message_timestamp = msg.getMonotonicTimestamp();
}
```

`uavcan.protocol.dynamic_node_id.Allocation`

Default data type ID: 1

```
#
# This message is used for dynamic Node ID allocation.
#
# When a node needs to request a node ID dynamically, it will transmit an anonymous message transfer
# In order to reduce probability of CAN ID collisions when multiple nodes are publishing this request
# field of anonymous message transfer includes a Discriminator, which is a special field that has to
# random data by the transmitting node. Since Discriminator collisions are likely to happen (probabil
# 0.006%), nodes that are requesting dynamic allocations need to be able to handle them correctly. He
# resolution protocol is defined (alike CSMA/CD). The collision resolution protocol is based on two r
# transmission intervals:
#
# - Request period - Trequest.
# - Follow up delay - Tfollowup.
#
# Recommended randomization ranges for these intervals are documented in the constants of this messag
# Random intervals must be chosen anew per transmission, whereas the Discriminator value is allowed t
# per node.
#
# In the below description the following terms are used:
# - Allocator - the node that serves allocation requests.
# - Allocatee - the node that requests an allocation from the Allocator.
#
# The response timeout is not explicitly defined for this protocol, as the Allocatee will request the
# Trequest units of time later again, unless the allocation has been granted. Despite this, the imple
# consider the value of FOLLOWUP_TIMEOUT_MS as an allocation timeout, if necessary.
#
# On the allocatee's side the protocol is defined through the following set of rules:
#
# Rule A. On initialization:
# 1. The allocatee subscribes to this message.
# 2. The allocatee starts the Request Timer with a random interval of Trequest.
#
# Rule B. On expiration of Request Timer:
# 1. Request Timer restarts with a random interval of Trequest.
# 2. The allocatee broadcasts a first-stage Allocation request message, where the fields are assigned
#    node_id                 - preferred node ID, or zero if the allocatee doesn't have any preferenc
#    first_part_of_unique_id - true
#    unique_id               - first MAX_LENGTH_OF_UNIQUE_ID_IN_REQUEST bytes of unique ID
#
# Rule C. On any Allocation message, even if other rules also match:
# 1. Request Timer restarts with a random interval of Trequest.
#
# Rule D. On an Allocation message WHERE (source node ID is non-anonymous) AND (allocatee's unique ID
# bytes available in the field unique_id) AND (unique_id is less than 16 bytes long):
# 1. The allocatee waits for Tfollowup units of time, while listening for other Allocation messages.
#    message is received during this time, the execution of this rule will be terminated. Also see ru
# 2. The allocatee broadcasts a second-stage Allocation request message, where the fields are assigne
```

```
#                             equal to number of bytes in the received unique ID
#
# Rule E. On an Allocation message WHERE (source node ID is non-anonymous) AND (unique_id fully matc
# unique ID) AND (node_id in the received message is not zero):
# 1. Request Timer stops.
# 2. The allocatee initializes its node_id with the received value.
# 3. The allocatee terminates subscription to Allocation messages.
# 4. Exit.
#

#
# Recommended randomization range for request period.
#
# These definitions have an advisory status; it is OK to pick higher values for both bounds, as it wo
# protocol compatibility. In fact, it is advised to pick higher values if the target application is r
# about the time it will spend on completing the dynamic node ID allocation procedure, as it will rec
# interference with other nodes, possibly of higher importance.
#
# The lower bound shall not be lower than FOLLOWUP_TIMEOUT_MS, otherwise the request may conflict wit
#
uint16 MAX_REQUEST_PERIOD_MS = 1000     # It is OK to exceed this value
uint16 MIN_REQUEST_PERIOD_MS = 600      # It is OK to exceed this value

#
# Recommended randomization range for followup delay.
# The upper bound shall not exceed FOLLOWUP_TIMEOUT_MS, because the allocator will reset the state or
#
uint16 MAX_FOLLOWUP_DELAY_MS = 400
uint16 MIN_FOLLOWUP_DELAY_MS = 0        # Defined only for regularity; will always be zero.

#
# Allocator will reset its state if there was no follow-up request in this amount of time.
#
uint16 FOLLOWUP_TIMEOUT_MS = 500

#
# Any request message can accommodate no more than this number of bytes of unique ID.
# This limitation is needed to ensure that all request transfers are single-frame.
# This limitation does not apply to CAN FD transport.
#
uint8 MAX_LENGTH_OF_UNIQUE_ID_IN_REQUEST = 6

#
# When requesting an allocation, set the field 'node_id' to this value if there's no preference.
#
uint7 ANY_NODE_ID = 0

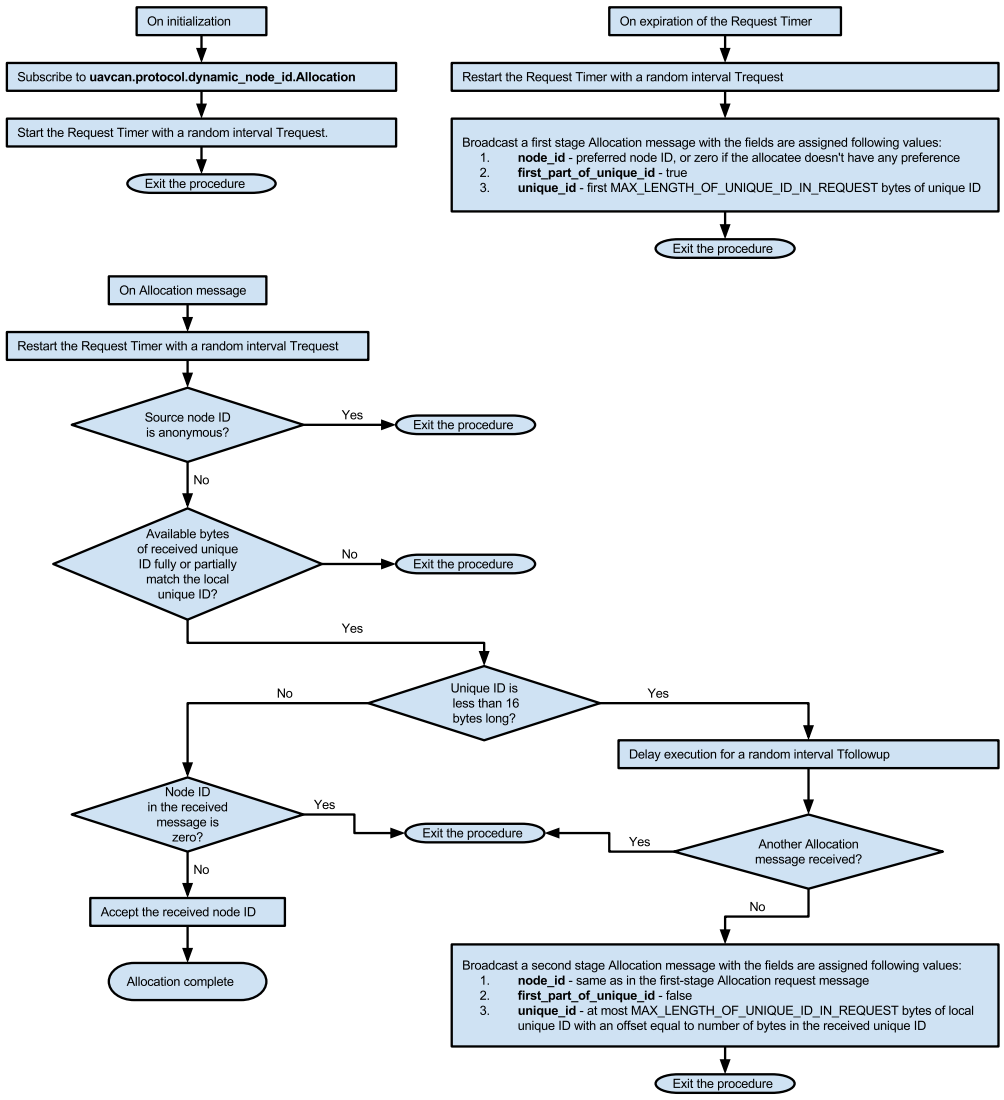
#
# If transfer is anonymous, this is the preferred ID.
# If transfer is non-anonymous, this is allocated ID.
#
# If the allocatee does not have any preference, this value must be set to zero. In this case, the al
# must choose the highest unused node ID value for this allocation (except 126 and 127, that are rese
# network maintenance tools). E.g., if the allocation table is empty and the node has requested an al
# without any preference, the allocator will grant the node ID 125.
#
# If the preferred node ID is not zero, the allocator will traverse the allocation table starting fro
# preferred node ID upward, until a free node ID is found. If a free node ID could not be found, the
# allocator will restart the search from the preferred node ID downward, until a free node ID is four
#
# In pseudocode:
#   int findFreeNodeID(const int preferred)
#   {
#       // Search up
#       int candidate = (preferred > 0) ? preferred : 125;
#       while (candidate <= 125)
#       {
#           if (!isOccupied(candidate))
#               return candidate;
#           candidate++;
#       }
#       // Search down
#       candidate = (preferred > 0) ? preferred : 125;
#       while (candidate > 0)
#       {
#           if (!isOccupied(candidate))
```

```
#       // Not found
#       return -1;
#   }
#
uint7 node_id

#
# If transfer is anonymous, this field indicates first-stage request.
# If transfer is non-anonymous, this field should be assigned zero and ignored.
#
bool first_part_of_unique_id

#
# If transfer is anonymous, this array must not contain more than MAX_LENGTH_OF_UNIQUE_ID_IN_REQUEST
# Note that array is tail-optimized, i.e. it will not be prepended with length field.
#
uint8[<=16] unique_id
```

The following diagram may aid understanding of the allocatee side of the algorithm (click to enlarge):



Example

The following log provides a real-world example of a dynamic node ID allocation process:

```
Time   CAN ID     CAN data field
1.117  1EEE8100   01 44 C0 8B 63 5E 05 C0
1.117  1E000101   00 44 C0 8B 63 5E 05 C0
1.406  1EEBE500   00 F4 BC 10 96 DF 11 C1
1.406  1E000101   05 B0 00 44 C0 8B 63 81
1.406  1E000101   5E 05 F4 BC 10 96 DF 21
1.406  1E000101   11 41
1.485  1E41E100   00 A8 BA 54 47 C2
1.485  1E000101   29 BA FA 44 C0 8B 63 82
1.485  1E000101   5E 05 F4 BC 10 96 DF 22
1.485  1E000101   11 A8 BA 54 47 42
```

First, the allocatee waits for a random time interval in order to ensure that other allocations are not happening at the moment. After the delay, the allocatee announces its intention to get a node ID allocation by broadcasting the following anonymous message:

The allocator responds immediately with confirmation:

```
1.117  1E000101   00 44 C0 8B 63 5E 05 C0
```

The allocatee waits for another random time interval in order to ensure that it will not conflict with other nodes that have unique node ID with the same first six bytes. After the delay, the allocatee sends the second-stage request:

```
1.406  1EEBE500   00 F4 BC 10 96 DF 11 C1
```

The allocator responds immediately with confirmation. This time, the confirmation contains 12 bytes of unique ID, so it doesn't fit one CAN frame, therefore the allocator resorts to a multi-frame transfer:

```
1.406  1E000101   05 B0 00 44 C0 8B 63 81
1.406  1E000101   5E 05 F4 BC 10 96 DF 21
1.406  1E000101   11 41
```

The allocatee waits for another random time interval in order to ensure that it will not conflict with other nodes that have unique node ID with the same first twelve bytes. After the delay, the allocatee sends the third-stage request:

```
1.485  1E41E100   00 A8 BA 54 47 C2
```

At this moment the allocator has received full unique ID and the preferred node ID of the allocatee as well. The allocator can carry out the actual allocation and send a response:

```
1.485  1E000101   29 BA FA 44 C0 8B 63 82
1.485  1E000101   5E 05 F4 BC 10 96 DF 22
1.485  1E000101   11 A8 BA 54 47 42
```

This completes the process. Next time the allocatee sends an allocation request, it will be provided with the same node ID.

The values used in the example above were the following:

Name	Value
Unique ID	`44 C0 8B 63 5E 05 F4 BC 10 96 DF 11 A8 BA 54 47` (hex)
Preferred node ID	0 (any)
Allocated node ID	125

Non-redundant allocator

DroneCAN does not impose specific requirements to the implementation of a non-redundant allocator, except its duties listed below.

Duties of the allocator

The allocator is tasked with monitoring the nodes present in the network. When a new node appears, the allocator must invoke `uavcan.protocol.GetNodeInfo` on it, and check the received unique ID against the allocation table. If a matching entry is not found in the table, the allocator will create one. If the node failed to respond to `uavcan.protocol.GetNodeInfo` after at least 3 attempts, the allocator will extend the allocation table with a *mock entry*, where the node ID is matching the real node ID of the non-responding node, and unique ID is set to zero. This ensures that dynamic nodes will not be granted a node ID value that is already taken by a static node. This requirement demonstrates why is it mandatory that dynamic nodes use the same unique ID both when responding to `uavcan.protocol.GetNodeInfo` and when publishing allocation requests.

Redundant allocators

Understandable Consensus Algorithm (Extended Version)" (Diego Ongaro and John Ousterhout). The following text assumes that the reader is familiar with the paper.

Raft log

The *Raft log* contains entries of type `uavcan.protocol.dynamic_node_id.server.Entry` (defined below), where every entry contains Raft term number, unique ID, and the matching node ID value. Therefore, the raft log is the allocation table itself.

Since the maximum number of entries in the allocation table is limited by the range of node ID, the log cannot contain more than 127 entries. Therefore, snapshot transfer and log compaction are not required, so they are not implemented in the algorithm.

When a server becomes the leader, it checks if the Raft log contains an entry for its own unique ID, and if it doesn't, the leader adds its own allocation entry to the log. This feature guarantees that the raft log always contains at least one entry, therefore it is not necessary to support negative log indices, as proposed by the Raft paper.

Since the log is write-only and limited in growth, all allocations are permanent. This restriction is acceptable, since DroneCAN is a vehicle bus, and configuration of vehicle's components is not expected to change frequently. Old allocations can be removed in order to free node IDs for new allocations, by clearing the Raft log on all allocators.

Cluster configuration

The allocators need to be aware of each other's node ID in order to form a cluster. In order to learn each other's node ID values, the allocators broadcast messages of type `uavcan.protocol.dynamic_node_id.server.Discovery` (defined below) until the cluster is fully discovered.

This extension to the Raft algorithm makes the cluster almost configuration-free - the only parameter that must be configured on all servers of the cluster is the number of nodes in the cluster (everything else will be auto-detected).

Runtime cluster membership changes are not supported, since they are not needed for a vehicle bus.

Duties of the leader

The leader is tasked with monitoring the nodes present in the network. Please refer to the section dedicated to duties of a non-redundant allocator for details.

Only the leader can process allocation requests and engage in communication with allocatees. An allocator is allowed to send allocation responses only if both conditions are met:

- The allocator is a leader.
- Its replica of the Raft log does not contain uncommitted entries (i.e. the last allocation request has been completed successfully).

The second condition needs to be explained by an example.

Consider a case with two Raft nodes that are residing in different network partitions, unable to communicate with each other - **A** and **B**, both of them are leaders; **A** can commit to the log, and **B** is in a minor partition. Then there is an allocatee **X** that can exchange with both leaders, and an allocatee **Y** that can exchange only with **A**. Such a situation can occur as a result of a specific failure mode of redundant interfaces.

Both allocatees **X** and **Y** initially send first-stage allocation requests; **A** responds to **Y** with a first-stage response, whereas **B** responds to **X**. Both **X** and **Y** will issue follow-up requests, which may cause **A** to mix allocation requests from different nodes, leading to reception of an invalid unique ID. When both leaders receive full unique ID values (**A** will receive an invalid one, and **B** will receive a valid unique ID of **X**), only **A** will be able to make a commit, because **B** is in a minor partition. Since both allocatees were unable to receive node ID values in this round, they will retry later.

Now, in order to prevent **B** from disrupting allocatee-allocator communication again, we introduce this second restriction: an allocator cannot exchange with allocatees as long as its log contains uncommitted entries.

Only CAN FD can offer perfectly reliable allocation exchanges.

`uavcan.protocol.dynamic_node_id.server.AppendEntries`

Default data type ID: 30

```
#
# THIS DEFINITION IS SUBJECT TO CHANGE.
#
# This type is a part of the Raft consensus algorithm.
# Please refer to the specification for details.
#

#
# Given min election timeout and cluster size, the maximum recommended request interval can be derive
#
#   max recommended request interval = (min election timeout) / 2 requests / (cluster size - 1)
#
# The equation assumes that the Leader requests one Follower at a time, so that there's at most one p
# at any moment. Such behavior is optimal as it creates uniform bus load, but it is actually implemen
# Obviously, request interval can be lower than that if needed, but higher values are not recommended
# cause Followers to initiate premature elections in case of intensive frame losses or delays.
#
# Real timeout is randomized in the range (MIN, MAX], according to the Raft paper.
#
uint16 DEFAULT_MIN_ELECTION_TIMEOUT_MS = 2000
uint16 DEFAULT_MAX_ELECTION_TIMEOUT_MS = 4000

#
# Refer to the Raft paper for explanation.
#
uint32 term
uint32 prev_log_term
uint8 prev_log_index
uint8 leader_commit

#
# Worst-case replication time per Follower can be computed as:
#
#   worst replication time = (127 log entries) * (2 trips of next_index) * (request interval per Foll
#
Entry[<=1] entries

---

#
# Refer to the Raft paper for explanation.
#
uint32 term
bool success
```

`uavcan.protocol.dynamic_node_id.server.RequestVote`

Default data type ID: 31

```
#
# THIS DEFINITION IS SUBJECT TO CHANGE.
#
# This type is a part of the Raft consensus algorithm.
# Please refer to the specification for details.
#

#
# Refer to the Raft paper for explanation.
#
uint32 term
uint32 last_log_term
uint8 last_log_index

---

#
# Refer to the Raft paper for explanation.
```

`uavcan.protocol.dynamic_node_id.server.Discovery`

Default data type ID: 390

```
#
# THIS DEFINITION IS SUBJECT TO CHANGE.
#
# This message is used by allocation servers to find each other's node ID.
# Please refer to the specification for details.
#
# A server should stop publishing this message as soon as it has discovered all other nodes in the cl
#
# An exception applies: when a server receives a Discovery message from another server where the list
# of known nodes is incomplete (i.e. len(known_nodes) < configured_cluster_size), the server must
# publish a discovery message once. This condition allows other servers to quickly re-discover the cl
# after restart.
#

#
# This message should be broadcasted by the server at this interval until all other servers are disco
#
uint16 BROADCASTING_PERIOD_MS = 1000

#
# Number of servers in the cluster as configured on the sender.
#
uint8 configured_cluster_size

#
# Node ID of servers that are known to the publishing server, including the publishing server itself
# Capacity of this array defines maximum size of the server cluster.
#
uint8[<=5] known_nodes
```

`uavcan.protocol.dynamic_node_id.server.Entry`

```
#
# THIS DEFINITION IS SUBJECT TO CHANGE.
#
# One dynamic node ID allocation entry.
# This type is a part of the Raft consensus algorithm.
# Please refer to the specification for details.
#

uint32 term             # Refer to the Raft paper for explanation.

uint8[16] unique_id     # Unique ID of this allocation.

void1
uint7 node_id           # Node ID of this allocation.
```

Example

The following log demonstrates relevant messages that were transferred over the CAN bus in the process of a dynamic node ID allocation for one allocatee, where the allocators were running a three-node Raft cluster. All node status messages were removed for clarity.

The configuration was as follows:

Name	Value
Number of allocators	3
Allocators' node ID	1, 2, 3
Leader's node ID	1
Allocatee's unique ID	`44 C0 8B 63 5E 05 F4 BC 83 3B 3A 88 1C 43 60 50` (hex)

Allocated node ID	125
Discovery broadcasting interval	1 second
AppendEntries interval	1 second per follower

```
Time   CAN ID     CAN data field
0.000  1E018601   03 01 C0
0.512  1E018602   03 02 01 C0
0.905  1E018603   03 03 01 02 C0
1.000  1E018601   03 01 02 03 C1
1.512  1E018602   03 02 01 03 C1
<cluster maintenance traffic omitted for clarity>
2.569  1EEE8100   01 44 C0 8B 63 5E 05 C0
2.569  1E000101   00 44 C0 8B 63 5E 05 C0
2.684  1E238D00   00 F4 BC 83 3B 3A 88 C1
2.684  1E000101   5C EF 00 44 C0 8B 63 81
2.684  1E000101   5E 05 F4 BC 83 3B 3A 21
2.684  1E000101   88 41
2.756  1E1E8381   5F CF 2E 00 00 00 04 85
2.756  1E1E8381   00 00 00 05 05 65
2.756  1E1E0183   2E 00 00 00 80 C5
2.871  1E63ED00   00 1C 43 60 50 C2
3.256  1E1E8281   9C 38 2E 00 00 00 04 87
3.256  1E1E8281   00 00 00 05 05 2E 00 27
3.256  1E1E8281   00 00 44 C0 8B 63 5E 07
3.256  1E1E8281   05 F4 BC 83 3B 3A 88 27
3.256  1E1E8281   1C 43 60 50 7D 47
3.258  1E1E0182   2E 00 00 00 80 C7
3.563  1E2F0D00   01 44 C0 8B 63 5E 05 C3
3.756  1E1E8381   9C 38 2E 00 00 00 04 86
3.756  1E1E8381   00 00 00 05 05 2E 00 26
3.756  1E1E8381   00 00 44 C0 8B 63 5E 06
3.756  1E1E8381   05 F4 BC 83 3B 3A 88 26
3.756  1E1E8381   1C 43 60 50 7D 46
3.756  1E000101   C7 36 FA 44 C0 8B 63 82
3.756  1E000101   5E 05 F4 BC 83 3B 3A 22
3.756  1E000101   88 1C 43 60 50 42
3.758  1E1E0183   2E 00 00 00 80 C6
4.256  1E1E8281   65 19 2E 00 00 00 2E 88
4.256  1E1E8281   00 00 00 06 06 68
4.256  1E1E0182   2E 00 00 00 80 C8
4.756  1E1E8381   65 19 2E 00 00 00 2E 87
4.756  1E1E8381   00 00 00 06 06 67
4.756  1E1E0183   2E 00 00 00 80 C7
```

Once the first node of the cluster has been started, it has published a cluster discovery message so it could become aware of its siblings, and other two nodes that were started a fraction of a second later did the same:

```
0.000  1E018601   03 01 C0

0.512  1E018602   03 02 01 C0

0.905  1E018603   03 03 01 02 C0

1.000  1E018601   03 01 02 03 C1

1.512  1E018602   03 02 01 03 C1
```

It can be seen that the last two discovery messages contain complete list of all nodes in the cluster. The allocators have detected the fact that all nodes in the cluster were now aware of each other, and ceased to broadcast discovery messages in order to not pollute the bus with redundant traffic.

Afterwards, the allocators ran elections and have elected the node 1 as their leader. The leader then performed a few AppendEntries calls in order to synchronize the replicated log. These exchanges are not shown for the sake of clarity.

The allocatee has appeared on the bus and has published first-stage and second-stage allocation requests. The current leader was in charge with communicating with allocatee, other two allocators

```
2.569  1EEE8100   01 44 C0 8B 63 5E 05 C0   <-- First stage request

2.569  1E000101   00 44 C0 8B 63 5E 05 C0   <-- First stage response

2.684  1E238D00   00 F4 BC 83 3B 3A 88 C1   <-- Second stage request

2.684  1E000101   5C EF 00 44 C0 8B 63 81   <-- Second stage response
2.684  1E000101   5E 05 F4 BC 83 3B 3A 21
2.684  1E000101   88 41
```

While the allocatee was waiting for expiration of the random timeout, the leader has performed a keep-alive AppendEntries call to the allocator 3:

```
2.756  1E1E8381   5F CF 2E 00 00 00 04 85   <-- Empty AppendEntries request
2.756  1E1E8381   00 00 00 05 05 65

2.756  1E1E0183   2E 00 00 00 80 C5         <-- AppendEntries response
```

Then the allocatee has broadcasted the third-stage allocation request:

```
2.871  1E63ED00   00 1C 43 60 50 C2
```

At this point the leader had the full unique ID of the allocatee, so it has started the process of allocation and log replication. In order to complete the allocation, the leader had to replicate the new entry of the Raft log to a majority of allocators (see the Raft paper for details):

```
3.256  1E1E8281   9C 38 2E 00 00 00 04 87   <-- AppendEntries request with new allocation
3.256  1E1E8281   00 00 00 05 05 2E 00 27
3.256  1E1E8281   00 00 44 C0 8B 63 5E 07
3.256  1E1E8281   05 F4 BC 83 3B 3A 88 27
3.256  1E1E8281   1C 43 60 50 7D 47

3.258  1E1E0182   2E 00 00 00 80 C7         <-- AppendEntries response with confirmation
```

It can be seen that the follower took 2 milliseconds to update its persistent storage.

While the leader was busy replicating the allocation table (it could not complete the allocation until the new log entry was committed), the allocatee has given up waiting for a response and decided to restart the process. This is not an error condition, but a normal behavior. This time the leader did not engage in communication with the allocatee, because the Raft log contained uncommitted entries.

```
3.563  1E2F0D00   01 44 C0 8B 63 5E 05 C3   <-- First stage request, no response from the leader
```

Some time later the leader decided to replicate the new log entry to the other follower:

```
3.756  1E1E8381   9C 38 2E 00 00 00 04 86   <-- AppendEntries request with new allocation
3.756  1E1E8381   00 00 00 05 05 2E 00 26
3.756  1E1E8381   00 00 44 C0 8B 63 5E 06
3.756  1E1E8381   05 F4 BC 83 3B 3A 88 26
3.756  1E1E8381   1C 43 60 50 7D 46
```

Immediately afterwards, the leader has noticed that the new entry has already been replicated to a majority of allocators, therefore (see the Raft paper) the commit index could be incremented, which completed the allocation. Having detected that, the leader has published the allocation response:

```
3.756  1E000101   C7 36 FA 44 C0 8B 63 82
3.756  1E000101   5E 05 F4 BC 83 3B 3A 22
3.756  1E000101   88 1C 43 60 50 42
```

While the leader was engaged in communications with the allocatee, the follower 3 has finished updating its persistent storage and responded with confirmation:

At this moment the process was finished. The leader then continued to invoke keep-alive AppendEntries calls to the followers:

```
4.256  1E1E8281   65 19 2E 00 00 00 2E 88   <-- Empty AppendEntries request
4.256  1E1E8281   00 00 00 06 06 68

4.256  1E1E0182   2E 00 00 00 80 C8         <-- AppendEntries response

4.756  1E1E8381   65 19 2E 00 00 00 2E 87   <-- Empty AppendEntries request
4.756  1E1E8381   00 00 00 06 06 67

4.756  1E1E0183   2E 00 00 00 80 C7         <-- AppendEntries response
```